

Report progetto di Scalable and Cloud Programming

Daniele Romanella - 0001170158 - daniele.romanella@studio.unibo.it

17 febbraio 2025

1 Approcci utilizzati

Lo sviluppo del progetto ha portato alla sperimentazione e all'utilizzo di diversi approcci per risolvere il problema.

1.1 Primo approccio

Il primo approccio utilizzato per risolvere il problema non ha fatto uso di Partitioner per capire quanto un partitioner avrebbe influenzato le prestazioni.

1.1.1 Funzionamento

Il primo approccio è la base di tutti gli approcci successivi e funziona nel modo seguente:

1. Il programma esegue il parsing del file CSV e fa il cast ad intero delle coppie (o, p) dove o è il numero dell'ordine e p l'identificativo del prodotto.
2. Il programma raggruppa per identificativo dell'ordine o il dataset di cui prima, quindi si ottiene un RDD del tipo:

$$o : Int \Rightarrow List[Int]$$

dove o è il numero dell'ordine e la lista a lui associata contiene tutti i prodotti p che sono stati acquistati in quell'ordine.

3. A questo punto il programma mantiene solo le liste di prodotti comprati insieme e crea le coppie di tutti i prodotti co-acquistati, utilizzando una flatMap tutte le coppie confluiscono in un'unica collezione.
4. Avendo una collezione con tante coppie $(p1, p2)$, coacquistati, possiamo effettuare su essa un groupBy e successivamente otteniamo un RDD con la struttura seguente:

$$(p1, p2) : (Int, Int) \Rightarrow List[(Int, Int)]$$

Dopodichè, per ottenere il risultato occorre fondere tutto in una collezione che contiene triple (x, y, z) dove $x, y = p1, p2$ e $z = \#List[(Int, Int)]$ ¹, ovvero la cardinalità della lista associata alla chiave $(p1, p2)$

1.1.2 Prestazioni al variare della dimensione del dataset

Il dataset dato ha una dimensione di circa 32 milioni di righe. In questo approccio e in quelli successivi il dataset è stato diviso in 5 sotto-frazioni: 1/32, 1/16, 1/8, 1/4, 1/2 e il dataset intero (indicato con 1). Possiamo osservare il tempo di esecuzione di questo approccio, insieme a tutti gli altri nella **Tabella 1**. Constatiamo che questo approccio è il peggiore nel caso in cui si ha un numero considerevole di righe nel file (oltre 8 milioni, 1/4 del dataset).

¹Cardinalità della lista

Tabella 1: Tempo di esecuzione (in secondi) dei vari approcci al variare della dimensione del dataset

Approccio	1/32	1/16	1/8	1/4	1/2	1
Primo	46	55	76	125	277	765
Secondo	59	62	79	98	135	186
Quarto	51	67	75	91	122	184

1.1.3 Prestazioni al variare dei nodi

Possiamo notare che al variare dei nodi il tempo di esecuzione non diminuisce, anzi aumenta. Analizzando i dati forniti dalla console di Dataproc, interrogando il Job eseguito, possiamo constatare che non tutti i processori vengono utilizzati simultaneamente, come mostrato nella **Figura 1**. Sprestando le risorse utilizzate per allocare più nodi. I tempi di esecuzione, al variare dei nodi impiegati nel job sono disponibili nella **Tabella 2**.

Nella **Tabella 2** notiamo che eseguendo il programma impiega addirittura meno tempo sfruttando solo 2 nodi. Proprio perchè non stiamo sfruttando tutta la potenza di calcolo che abbiamo a disposizione nelle esecuzioni con 3 e 4 nodi.

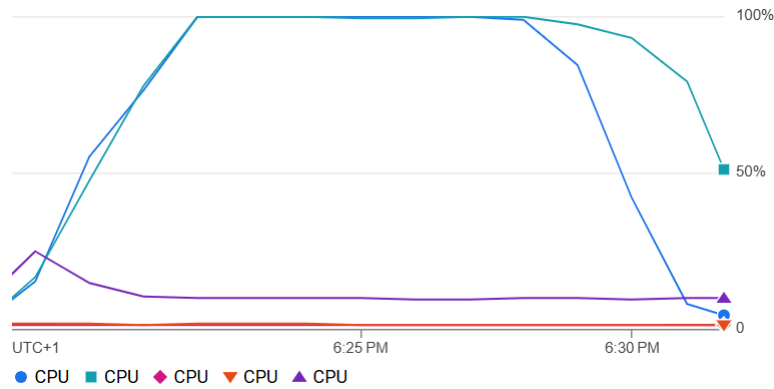


Figura 1: Utilizzo delle varie CPU del primo approccio.

Tabella 2: Tempo di esecuzione (in secondi) dei vari approcci al variare del numero di nodi

Approccio	2 nodi	3 nodi	4 nodi
Primo	720	778	765
Secondo	246	206	186
Quarto	232	196	184

1.2 Secondo approccio

Il secondo approccio è una variazione del primo, l'unica modifica apportata è l'utilizzo di un partitioner custom.

Il partitioner divide gli ordini in base al modulo per il numero di partizioni, quindi tutti i prodotti associati allo stesso ordine sono nella stessa partizione.

1.2.1 Numero di partizioni

Il numero di partizioni è un parametro critico: occorre conoscere il cluster su cui sarà eseguito il programma. Analizzando la Google Cloud platform ho constatato che tutti i worker hanno 4 vCPU.

Essendo che il programma deve essere eseguito su 2, 3 e 4 nodi, allora ho scelto 96 come numero di partizioni perchè divisibile per 8, 12 e 16.

E' stato osservato empiricamente che avere un numero di partizioni divisibile per il numero di core disponibili dà un gain prestazionale se le partizioni sono ben distribuite e il carico di lavoro per ogni partizione è simile.

1.2.2 Prestazioni

Le prestazioni di questo approccio al variare del dataset e del numero di nodi sono osservabili nelle tabelle **Tabella 1** e **Tabella 2**. A differenza del primo approccio, notiamo che al variare dei nodi il tempo di esecuzione diminuisce grazie al partizionamento dei dati e allo sfruttamento di tutti i core allocati.

1.3 Terzo approccio

Il terzo approccio differisce da quelli precedenti per quanto riguarda il conteggio delle coppie (p_1, p_2) . Dopo aver ottenuto le coppie dei prodotti co-acquistati per ogni ordine, viene creata una Map mutabile del tipo

$$Map((Int, Int), Int)$$

Che conta tutte le occorrenze della chiave. Successivamente viene fatta una reduce di tutte le mappe di cui sopra e per tutte le chiavi che sono contenute in entrambe le Map viene fatta la somma. Un ulteriore passaggio è necessario per ottenere l'output come da specifica.

1.3.1 Prestazioni

Le prestazioni di questo approccio sono le peggiori. Nel caso del dataset intero (circa 32 milioni di righe) il programma *non termina* in 8 ore. ² Probabilmente le prestazioni sono impattate molto dalla reduce in cui vengono sommate le Map e dalle operazioni di intersezione fra gli insiemi delle chiavi.

1.4 Quarto approccio

Il quarto approccio, il migliore a livello prestazionale. Come possiamo constatare dalle tabelle **Tabella 1** e **Tabella 2**, osserviamo che è più veloce del primo e del secondo approcci utilizzati. Questo approccio si basa sull'implementazione del secondo. E consiste di utilizzare le funzioni che propagano i *partitioner*.

E' stato sufficiente modificare la funzione *flatMap* con *flatMapValues* e successivamente eseguire delle operazioni per ottenere un'unica collezione di coppie per poi raggrupparle e contarle.

2 Scalabilità

L'analisi di scalabilità è stata effettuata analizzando il quarto approccio ³. Procediamo ad eseguire le operazioni di analisi della scalabilità

2.1 Speedup

Lo speedup è il rapporto tra l'esecuzione del programma con un certo input x fissato, è definito come

$$S(n) = \frac{T(1)}{T(n)}$$

Nel caso in cui $n = 4$ allora:

$$S(4) = \frac{T(1)}{T(4)} = \frac{468}{184} \approx 2,54$$

Dunque, essendo $2,5 < n = 4$ sappiamo che lo speedup non è lineare.

Avendo a disposizione i dati per $n = 2$ e $n = 3$ possiamo calcolare lo speedup anche per 2 e 3 nodi.

$$S(2) = \frac{T(1)}{T(2)} = \frac{468}{235} \approx 1,99$$

$$S(3) = \frac{T(1)}{T(3)} = \frac{468}{196} \approx 2,39$$

²Le prestazioni sono analizzabili nel file di Google Fogli citato in calce al documento.

³Il migliore in termini prestazionali.

2.2 Strong Scaling Efficiency

La Strong Scaling Efficiency ci aiuta a calcolare l'impatto della legge di Amdahl sull'esecuzione del programma e a quantificare quanto impatta l'aggiunta di un nuovo nodo sul tempo di esecuzione. La Strong Scaling Efficiency è definita come segue:

$$E(n) = \frac{S(n)}{n}$$

Calcoliamo $E(n)$ per $n \in 1, 2, 3, 4$

$$E(1) = \frac{S(1)}{1} = \frac{1}{1} = 1$$

$$E(2) = \frac{S(2)}{2} \approx \frac{1,99}{2} \approx 0,995$$

$$E(3) = \frac{S(3)}{3} \approx \frac{2,39}{3} \approx 0,797$$

$$E(4) = \frac{S(4)}{4} \approx \frac{2,54}{4} \approx 0,635$$

Possiamo constatare l'andamento dell'efficienza osservando la **Figura 2**, la non speedup fraction si aggira a circa il 20%.

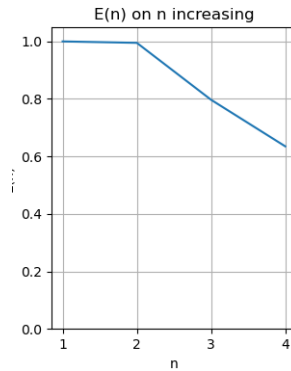


Figura 2: Strong Scaling Efficiency al variare dei nodi

2.3 Weak Scaling Efficiency

La Weak Scaling Efficiency ci consente di capire se, al variare del numero di processori e al numero di unità di computazione è possibile svolgere lo stesso lavoro nello stesso lasso di tempo oppure se occorre aspettare un tempo maggiore.

La Weak Scaling Efficiency è definita come

$$W(n) = \frac{T_1}{T_n}$$

Dove T_n indica il tempo di esecuzione di n unità di processo sfruttando n processori.

Se $W(n) = 1$ per un qualche n allora, il programma starà utilizzando efficientemente le risorse di calcolo allocate, in quanto, il tempo per eseguire i due programmi con n unità di calcolo e n processori sarà lo stesso di eseguire una sola unità di calcolo con un solo processore.

Quest'analisi prevede:

- Un'unità di calcolo è un worker su Dataproc
- Il lavoro destinato ad ogni nodo è di 8 milioni di righe, quindi in input, al programma sarà fornito un file di dimensione 8 milioni di righe * n

I risultati per $n \in 2, 3, 4$ sono:

$$W(2) = \frac{T_1}{T_2} = \frac{122}{145} \approx 0,84$$

$$W(3) = \frac{T_1}{T_3} = \frac{122}{176} \approx 0,69$$

$$W(4) = \frac{T_1}{T_4} = \frac{122}{184} \approx 0,66$$

Risultati più distanti da 1 ci dicono che le risorse di calcolo sono impiegate per gestire la concorrenza e la comunicazione fra nodi anziché essere impiegate per la risoluzione del problema.

3 Risorse utili

I tempi di esecuzione sono stati conservati in uno spreadsheet condiviso [raggiungibile da qui](#).

Inoltre, il repository github è disponibile [qui](#).

In seguito all'analisi effettuata si osserva che i prodotti 13176 e 47209 sono i più co-acquistati, in quanto acquistati insieme in 62341 ordini.